BZ205 Konu Anlatımı

Karmaşıklık Analizi

Hocanın ders videoları, slaytları ve sınav soruları temel alınmıştır

27 Ekim 2026

2 Karmaşıklık Analizi

2.1 Tanım ve amaç

Karmaşıklık analizi, bir algoritmanın *kodlanmadan önce* davranışının teorik olarak ortaya konmasıdır; genelde kaba koddan veya akış diyagramından yapılır. Aynı işi yapan algoritmaları karşılaştırmak (hız veya bellek açısından) için kullanılır. Kodlayıp ölçmek hem zaman alır hem de yanıltıcıdır: farklı makine, RAM, programlama dili ve programcı tecrübesi sonucu etkiler (*"fizikteki eşit koşullarda deney gibi"*).

2.2 Dört terim

Yürütme zamanı $T(n)$	Algoritmanın işini yapması için temel kabul edilen işlemlerden (atama, karşılaştırma, toplama, artırma, return, yazdırma…) *kaç tane* yapılması gerektiğini veren bağıntı. Girdi birden fazla ise $T(n, m)$.
Zaman karmaşıklığı	Asimptotik notasyona göre karmaşıklık *mertebesi* (davranışı): $O(n)$, $O(n^2)$, $O(\log n)$…
Alan maliyeti	Algoritmanın çalışması için gereken bellek miktarı.
Alan karmaşıklığı	Eleman sayısı büyüdükçe bellek gereksiniminin artış mertebesi.

! Hoca derste vurguladı [video 3 Karmaşıklık 06:09]

"Sınavlarda vesaire en çok karıştırdığımız şey bu. Yürütme zamanı dediğimiz zaman neyi istiyoruz? Zaman karmaşıklığı dediğimiz zaman neyi istiyoruz? Verilen cevaplar arasındaki en büyük karmaşa bu. ***Eğer karıştırılırsa bu konuyu bilmiyorsunuz demektir.****"*
Yürütme zamanı bir *fonksiyondur* ($T(n) = 3n + 4$); zaman karmaşıklığı bir *mertebedir* ($O(n)$).

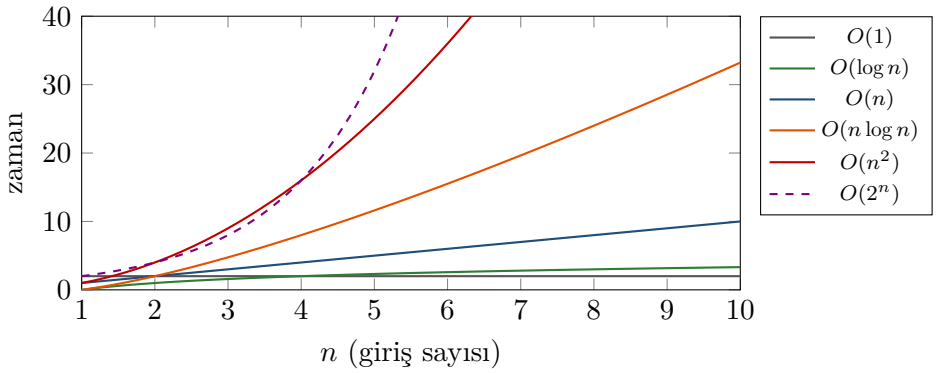
2.3 Asimptotik notasyonlar

- O ("order of") – **en kötü durum**, üst sınır: algoritma bundan daha kötü çalışamaz. *Genelde hesaplanan budur.*
- Ω – **en iyi durum**, alt sınır: bundan daha iyi çalışamaz.
- Θ – **ortalama durum**. Ders kapsamında: *"en iyi ve en kötü durum zaman karmaşıklıklarının derecesi aynıysa ortalama durumdan söz edebiliriz"* (ör. $3n + 1$ ve $4n \Rightarrow \Theta(n)$). Dereceler farklıysa (n ve n^2) Θ verilmez.

! Hoca derste vurguladı [video 3 Karmaşıklık 22:52 ve 41:28]

"Soruda verilen algoritmanın karmaşıklık analizini yapın derse burada en kötü durumu soruyor demektir. İllaki en kötü durum nedir diye sormasına gerek yok."
"Bir kısmınız şöyle yazıyor: $O = O(n^2)$. Öyle bir şey olmaz." Doğrusu: $T(n) = 3n^2 + \ldots$, zaman karmaşıklığı $O(n^2)$.

2.4 Bilinen büyüme fonksiyonları



Mertebe	Nerede karşımıza çıkar
$O(1)$ sabit	ekrana tek değer yazma, dizide k. elemana erişim (findKth), yığına push/pop, bağlantılı listede başa ekle
$O(\log n)$	ikili arama ($n = 2^h \Rightarrow h = \log_2 n$), her turda indisi yarıya bölen döngü, dengeli ağaçta arama
$O(n)$	tek döngü (1'den n'e), doğrusal arama, listeyi yazdırma
$O(n \log n)$	birleşmeli, kümeleme, (iyi durumda) hızlı sıralama
$O(n^2)$	iç içe iki döngü; kabarcık, seçmeli, araya sokma; hızlı sıralama en kötü
$O(n^3)$	üç iç içe döngü; `for i=1..n²` içinde `for j=1..n`
$O(2^n)$, $O(n!)$	bazı özyinelemeli (recursive) fonksiyonlar

Dikkat

"For görünce direkt doğrusalı veya kareseli yapıştırmamak lazım" – döngünün kaçtan kaça gittiğine bakın: `for(i=1;i<=n*n;i++) for(j=1;j<=n;j++)` $\Rightarrow n^3$; iç döngü `j=1..1` ise doğrusal; indis her turda ikiye bölünüyorsa logaritmik.

2.5 Sayma kuralları

1. **Değişken tanımları/ilk atamalar ihmal edilir** ve bu yazılır: "değişken tanımlamaları ihmal edilmiştir". İhmal edilmezse sadece sabit terim değişir.
2. **for(k=0; k<n; k++):** atama 1 kez, kontrol $n+1$ kez ($k = 0..n$), artırma n kez $\Rightarrow 2n+2$. Gövdeye n kez girilir.
3. **for(k=1; k<n; k++):** atama 1, kontrol n, artırma $n-1 \Rightarrow 2n$. Gövdeye $n-1$ kez girilir.
4. İç döngü maliyeti, dış döngünün kaç kez çalıştırdığıyla *çarpılır*. İç döngünün sınırı dış indise bağlıysa ($k < i$) toplam alınır: $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$.
5. Koşulda `&&` ile bağlanmış **iki kriter** varsa her kontrol **2 işlem** sayılır: *"Burada iki tane kriter var…ben buradaki maliyet ne bulursam 2 ile çarpmam lazım"* [video 5 Sıralama 2 33:49] (araya sokmanın iç döngüsü).
6. if kontrolü her zaman sayılır; if'in *içi* en kötü durumda her seferinde, en iyi durumda hiç çalışmayabilir.

7. Fonksiyon içinden başka fonksiyon çağrılıyorsa onun maliyeti ayrıca eklenir; return 1 işlemdir.

2.6 Derste çözülen örnekler

1) Dizinin ortalaması

```
float ortalama(int A[], int n) {
  float ort, toplam = 0;   // ihmal
  int k;                   // ihmal
  for (k = 0; k < n; k++)
     toplam += A[k];
  ort = toplam / n;
  return ort;
}
```

Satır	Tekrar
for $(1 + (n+1) + n)$	$2n + 2$
`toplam += A[k]`	n
`ort = toplam/n`	1
`return`	1
$T(n)$	$\mathbf{3n + 4}$

En iyi = en kötü $\Rightarrow O(n)$, $\Omega(n)$, $\Theta(n)$. Grafikte: $n = 0$'da 4 birimden başlayan eğimi 3 olan doğru.

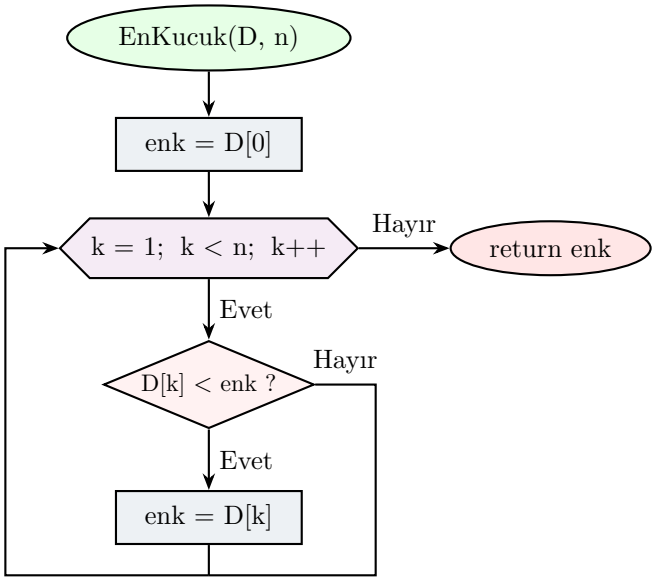
2) Dizideki en küçük eleman

```
int enkucuk(int A[], int n) {
  int enk, k;              // ihmal
  enk = A[0];
  for (k = 1; k < n; k++)
     if (A[k] < enk)
        enk = A[k];
  return enk;
}
```

Satır	En kötü	En iyi
`enk=A[0]`	1	1
for	$2n$	$2n$
if	$n - 1$	$n - 1$
`enk=A[k]`	$n - 1$	0
return	1	1
$T(n)$	$\mathbf{4n}$	$\mathbf{3n + 1}$

En kötü: dizi *büyükten küçüğe* sıralı (if içine her seferinde girer). En iyi: dizi *küçükten büyüğe* sıralı veya tüm elemanlar eşit. $O(n)$, $\Omega(n)$, $\Theta(n)$.

Akış diyagramı: en küçük eleman



En büyük eleman için < yerine > yazılır.

3) İki matrisin toplamı ($n \times m$)

```
for (i = 0; i < n; i++)          // 2n + 2
   for (j = 0; j < m; j++)       // n(2m + 2)
      C[i][j] = A[i][j] + B[i][j];   // n*m
```

$T(n,m) = 2n+2+2nm+2n+nm = \mathbf{3nm+4n+2}$. En kötü durum kare matris ($n=m$): $O(n^2)$ (karesel). En iyi durum *sütun vektörü* ($m=1$): $T = 7n+2 \Rightarrow \Omega(n)$. Dereceler farklı $\Rightarrow \Theta$ yok.

! Hoca derste vurguladı [video 4 Sıralama 1 34:21]

"İllaki sadece koşullu durumlarda en iyi durum oluşacak diye bir şey yok. Verinin geliş şekli farklı olursa da sonuç farklı olur."

! Hoca derste vurguladı Toplama + atama tek işlem mi? [video 4 Sıralama 1 25:32]

"Dışarıdan giriyorum. Buradan n kez çalışır. Dıştaki fordan. İçerideki for da m kez girer. O zaman şuradaki C'nin maliyeti $n \cdot m$."

`C[i][j] = A[i][j] + B[i][j]` satırında toplama *ve* atama olmasına rağmen hoca satırı **tek işlem** sayar (nm, $2nm$ değil); sonuç bu yüzden $3nm+4n+2$'dir. Aritmetik ortalamadaki `toplam += A[k]` de aynı şekilde n sayılır. Kural: **basit ifade satırı = 1 birim** × çalışma sayısı; yalnızca `for` başlığı 3 parçaya ayrılır (atama 1, kontrol $n+1$, artırma n). Ayrı sayılsa bile yalnız katsayı değişir ($4nm$), derece ve $O(nm)$ aynı kalır.

4) Özyinelemeli (recursive) faktöriyel

```
int fakt(int n) {
   if (n <= 1)            // n kez kontrol
      return 1;           // 1 kez
   else
      return n * fakt(n-1);   // n-1 kez
}
```

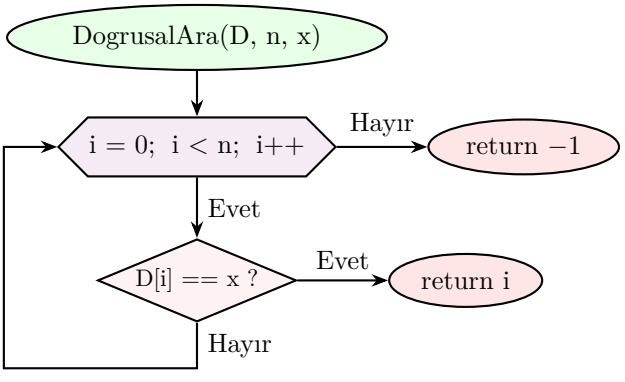
$T(n) = n+1+(n-1) = \mathbf{2n} \Rightarrow O(n)$. Döngülü sürümle aynı zaman karmaşıklığı; fakat **alan maliyeti yüksek**: her çağrı fonksiyonun yeni bir kopyasını oluşturur (5! için 4–5 kopya). Aynı durum birleşmeli ve hızlı sıralama için de geçerlidir.

5) Doğrusal (sıralı) arama

```
DogrusalAra(D, n, x)
  for (i = 0; i < n; i++)
     if (D[i] == x)
        return i
  return -1          // bulunamadi
```

Sırasız dizide baştan sona bakılır. En kötü durum (aranan sonda veya dizide yok): for $2n+2$, if n, return $1 \Rightarrow T(n) = 3n+3 \Rightarrow O(n)$. En iyi durum (aranan ilk elemanda): sabit sayıda işlem $\Rightarrow \Omega(1)$.

Akış diyagramı



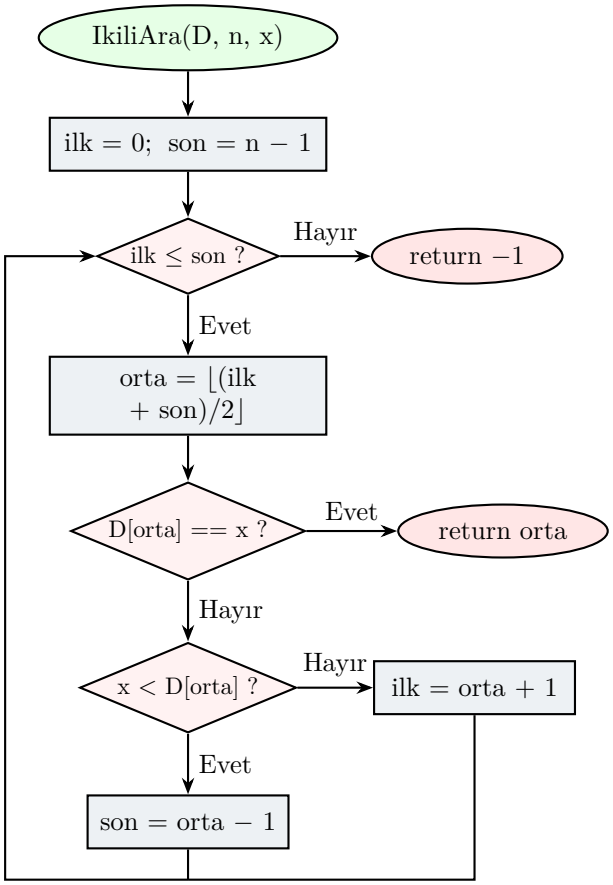
6) İkili arama neden $\log n$?

Sıralı dizide ortadaki eleman ile karşılaştırılır, yarısı elenir. h karşılaştırma sonunda tek eleman kalırsa $n = 2^h \Rightarrow \log_2 n = h$. Dolayısıyla $O(\log n)$. (Verinin sıralanma maliyeti ayrıdır.)

```
IkiliAra(D, n, x)
  ilk = 0;  son = n - 1
  while (ilk <= son)
     orta = (ilk + son) / 2
     if (D[orta] == x) return orta
     if (x < D[orta]) son = orta - 1
     else             ilk = orta + 1
  return -1
```

Her adımda dizinin yarısı elenir (hoca, video 3: 24:34). Dizinin sıralı olması gerekir; sıralama maliyeti ayrıdır.

Akış diyagramı



▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Arama algoritmaları: zaman ve alan karmaşıklığı

Durum	Doğrusal arama	İkili arama	Not
En iyi (Ω)	$\Omega(1)$	$\Omega(1)$	aranan ilk bakılan yerde (ilk eleman / ortadaki)
En kötü (O)	$O(n)$	$O(\log n)$	aranan sonda veya yok / tek eleman kalana kadar yarılama
Ortalama (Θ)	–	–	en iyi ve en kötü derecesi farklı $\Rightarrow \Theta$ verilmez
Ek alan	$O(1)$	$O(1)$	sadece birkaç indis değişkeni (iteratif sürüm)
Ön koşul	yok	dizi sıralı olmalı	sıralama maliyeti ayrıca eklenir

2.7 Sınavda yürütme zamanı nasıl yazılır? (şablon)

1. Kodu/kaba kodu satır numaralarıyla yazın; tanımlamalar için "ihmal" notu düşün.
2. Her satırın yanına kaç kez çalıştığını yazın (tablo). Döngüler için atama/kontrol/artırma ayrı ayrı; gövdedeki basit satırlar (ör. `C[i][j] = A[i][j] + B[i][j]`) tek işlem.
3. İç içe döngülerde çarpın veya ardışık toplam formülünü ($\frac{n(n+1)}{2}$) kullanın.
4. En kötü durum girdisini (ör. ters sıralı dizi) ve en iyi durum girdisini söyleyin; if içini buna göre sayın.

5. Toplayıp $T(n)$'i polinom olarak yazın; en büyük dereceli terim $\Rightarrow O(\cdot)$ / $\Omega(\cdot)$; dereceler eşitse $\Theta(\cdot)$.

Mazeret sınavında sorulan kod tipi

2024–25 mazeret sınavında "aşağıdaki fonksiyonların yürütme zamanlarını ve zaman karmaşıklıklarını hesaplayınız" diye `while` içinde iç içe `for` (`k<i`) ve `i<n*n` döngüleri verildi. Çözümü: `02_Vize_Cozumleri.pdf`, $T(n) = \frac{3}{2}n^3 + \frac{7}{2}n^2 + 4n + 2$ ve $T(n) = \frac{3}{2}n^4 + \frac{5}{2}n^2 + 3n + 4$.